



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

Tecnología Digital VI: Inteligencia Artificial

Taller de tuneo de arquitectura

Clase práctica 15

Motivación

- En caso de que no tengamos una **GPU o TPU** a mano, ¿cómo podemos usar una **herramienta online** para **entrenar** redes neuronales densas?
- A modo de práctica para el TP3, ¿cómo podemos **experimentar** con varias **arquitecturas** de redes neuronales?
- ¿Cómo podemos usar una **herramienta** online para **monitorear** el progreso de nuestros **experimentos**?

Herramientas | TPU

Una **TPU** (unidad de procesamiento de tensores, del inglés *tensor processing unit*) es un procesador diseñado específicamente para el procesamiento de tareas de inteligencia artificial que requieren grandes cantidades de operaciones de **multiplicación de matrices**.

A **diferencia de las CPUs o GPUs**, diseñadas para realizar una amplia gama de tareas; las TPU están altamente optimizadas para realizar operaciones matemáticas en grandes conjuntos de datos en paralelo. Esto es algo que las convierte en una solución ideal para realizar tareas de reconocimiento de voz, reconocimiento de imágenes o traducción de idiomas.

Herramientas | Monitoreo de progreso

Los **tiempos de entrenamiento** de las redes neuronales suelen ser largos y, como hay muchas opciones para construirlas (además de los hiperparámetros, debemos establecer cuántas capas queremos, cuántas neuronas poner en cada una, qué funciones de activación usar, etc.), suele ser necesario tener una forma visual de comparar los experimentos empíricos que hagamos, para ver su *performance*. Esto usualmente involucra monitorear, al menos, la **pérdida en entrenamiento**, la **pérdida en validación** y la **métrica de nuestro interés** (por ejemplo, *accuracy*).

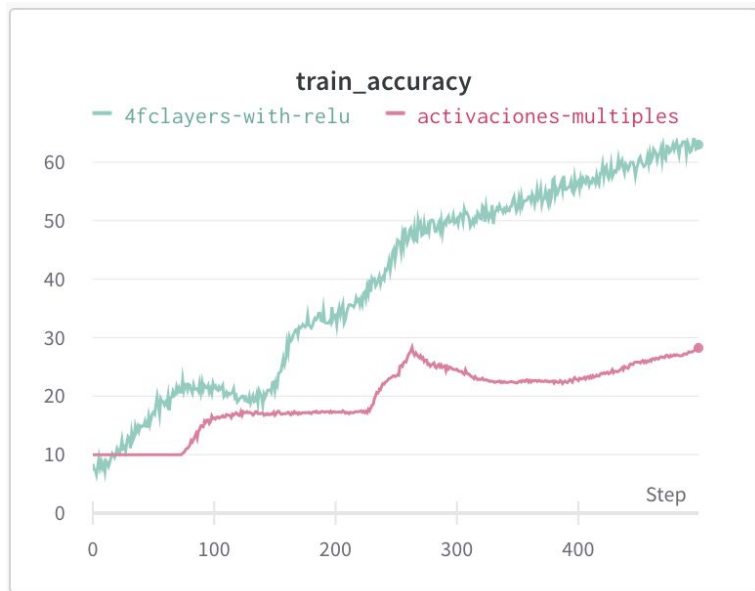
Una herramienta online muy valiosa que podemos usar es [Weights & Biases](#).

Herramientas | Weights & Biases | Qué es

Weights & Biases es una herramienta online que nos permite cargar todas las métricas de nuestros experimentos y compararlas visualmente.

Tiene una **API** que permite cargar los datos que querramos.

Su uso personal es **gratuito** y brinda hasta **100 GB** de almacenamiento para datos.



Herramientas | W&B | Pasos para usarlo

1. **Registrarse** [acá](#) (seleccionar **Personal** como modo de uso).
2. Copiar la **clave** correspondiente a la **API** (en **Home** dentro del recuadro amarillo o **Menu >> User Settings >> Danger zone >> API key**).
3. **Pegarla** en la *notebook* de Python.
4. Instalar la librería **wandb**.
5. Definir nombre del **proyecto**, nombre del **experimento** e **hiperparámetros** a guardar.
6. Usar los métodos de la **API** de **W&B**.

Veamos cómo hacer todo esto [en un notebook](#).

Herramientas | W&B | API vía Python

wandb.login(): realiza el login

wandb.init(): inicializa la ejecución. Sus parámetros usuales son los siguientes.

- **project**: contiene el nombre del proyecto; esto aparecerá en la interfaz de W&B.
- **entity**: refiere al nombre de usuario; no es obligatorio pero evita que “se pisen” si otra persona se logueó manualmente con la línea de comandos de W&B.
- **name**: nombre de la ejecución o del experimento; es lo que identificará al *run* específico.
- **config**: diccionario con hiperparámetros y metadata; se usa para loguear los datos que necesitaremos consultar en el futuro.

Herramientas | W&B | API vía Python (cont.)

wandb.log(): este método loguea propiamente. Se recomienda ejecutarlo una vez por *epoch* (o por el período que queramos loguear) y una vez al final, si deseamos loguear algo general; por ejemplo, el tiempo de ejecución del run.

`wandb.log()` tiene un parámetro llamado **commit** que, si se le pasa `False`, no enviará las métricas hasta que se llame pasándole `True`. Sirve por si deseamos llamar varias veces a `wandb.log()` por epoch pero sólo loguear una vez.

wandb.finish(): debe llamarse al final de la ejecución, para que el run sea marcado como finalizado en W&B.

La documentación oficial se encuentra disponible [acá](#).

Htas. | W&B | Beneficios y buenas prácticas

Beneficios:

- Gráficos **durante la ejecución** .
- Guardar toda la información de nuestros experimentos de forma **organizada**, en un lugar **centralizado** y de fácil **acceso**.

Buenas prácticas:

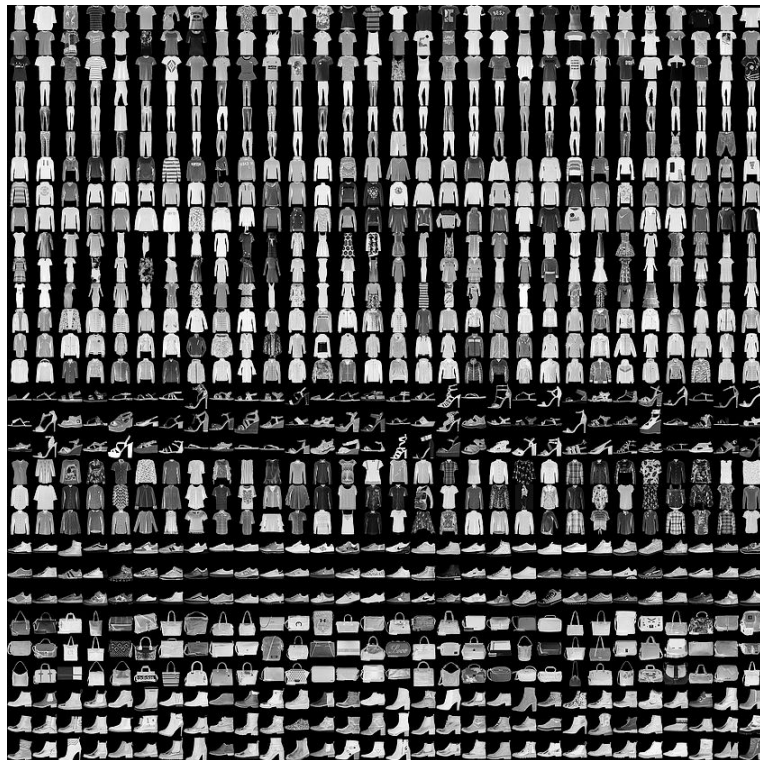
- Poner a los experimentos **nombres** que sean **descriptivos**.
- Guardar **todos los hiperparámetros** para posibles consultas futuras.

Taller | Datos

[Fashion-MNIST](#) es un *dataset* **balanceado** de imágenes de ropa con:
10 categorías, **60.000** imágenes de *train* y **10.000** imágenes de *test*.

Las imágenes son de **28x28** y en escala de **grises**.

Está integrado a **PyTorch** de forma *built-in* ([documentación oficial](#)).



Taller | Consignas

Ahora, vamos a usar **Google Colab** y **W&B** para entrenar un modelo que pueda **predecir a qué categoría pertenece una imagen de ropa** del dataset Fashion-MNIST.

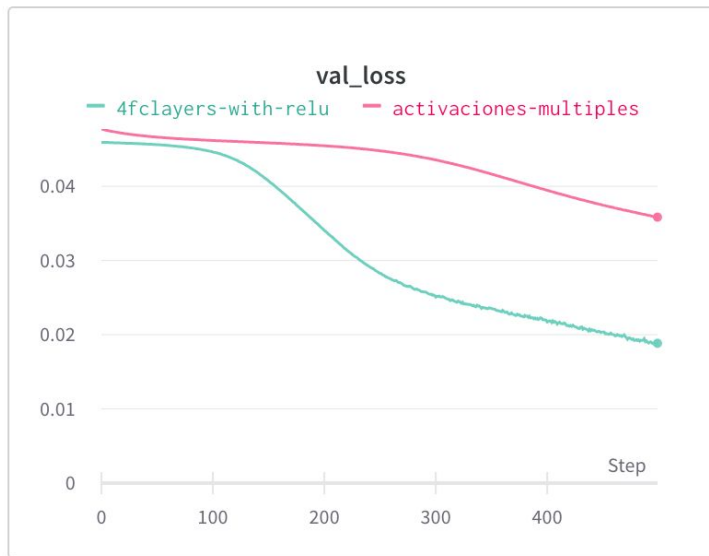
El objetivo del taller será:

- 1) Implementar el **logueo de métricas** en W&B
- 2) Realizar **experimentos** para mejorar la performance

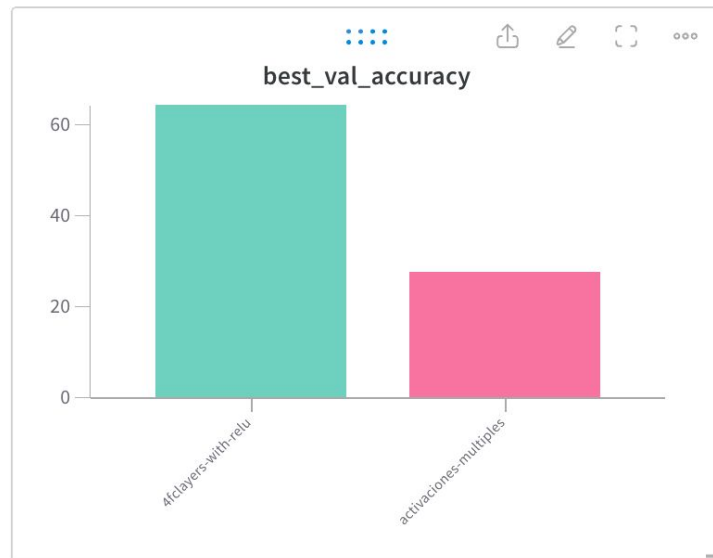
El uso de **Colab** es **optativo**; pero recomendado, si no cuentan con una GPU.

Veamos `td6-p15-b-consignas.ipynb`.

Taller | Sugerencias para W&B



Usar **line plots** para el **accuracy** y las **losses**.



Usar **bar charts** para el **training time** y el **best test accuracy**.

Taller | Ideas para tunear la arquitectura

- ❑ Cambiar la cantidad de **capas ocultas**.
- ❑ Cambiar la cantidad de **neuronas** de, al menos, una de las capas ocultas.
- ❑ Cambiar la cantidad de **épocas**.
- ❑ Cambiar el **tamaño de los lotes**.
- ❑ **Preprocesar** las imágenes (por ejemplo, estandarizar sus intensidades).
- ❑ Usar, al menos, una técnica de **regularización**, aplicada a redes neuronales, para lidiar con potencial *overfitting*, como *early stopping*, *weight decay*, *dropout* o *data augmentation* (por ejemplo, *random flip*).

Taller | Ideas para tunear la arquitectura (cont.)

Más detalladamente:

- **Early stopping.** Después de n cantidad de iteraciones en que el **error en validación no se redujo**, dejar de entrenar.
- **Weight decay.** Evitar que existan ws muy grandes; es decir, **evitar darle demasiado peso a alguna neurona**, que a fin de cuentas representa un *feature*. Lo hace sumando a la función de pérdida o la **norma L_1** (regularización lasso) o el **cuadrado de la norma L_2** (regularización ridge).
- **Dropout.** Con el **mismo fin que weight decay** (i.e., evitar que el modelo asigne demasiada responsabilidad a alguna neurona), aleatoriamente con cierta probabilidad determinada por nosotros de antemano (i.e., hiperparámetro), en cada iteración del algoritmo del gradiente descendente, **ignora neuronas** en pasada *forward* durante el entrenamiento.
- **Aumentación de datos.** Para incrementar el poder de generalización, al momento de entrenar, aumentar **artificialmente** la cantidad de observaciones aplicando transformaciones (e.g., escalar, trasladar, rotar, voltear, agregar ruido) en los datos (cuidadosamente; e.g., hemisferios del cerebro, gato sentado) y conservando las etiquetas.

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

$$L_1 \text{ norm} = \|x\|_1 = \sum_{i=1}^n |x_i|$$

$$L_2 \text{ norm} = \|x\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

L1 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^n |W_i|$$

L2 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^n W_i^2$$

Taller | Ideas para tunear la arquitectura (cont.)

- ❑ Cambiar la cantidad de **capas ocultas**.
- ❑ Cambiar la cantidad de **neuronas** de, al menos, una de las capas ocultas.
- ❑ Cambiar la cantidad de **épocas**.
- ❑ Cambiar el **tamaño de los lotes**.
- ❑ **Preprocesar** las imágenes (por ejemplo, estandarizar sus intensidades).
- ❑ Usar, al menos, una técnica de **regularización**, aplicada a redes neuronales, para lidiar con potencial *overfitting*, como *early stopping*, *weight decay*, *dropout* o *data augmentation* (por ejemplo, *random flip*).
- ❑ Usar **subsets** para entrenar más rápidamente, al elegir hiperparámetros.
- ❑ Usar **capas convolucionales** (no en este taller; a ver en próximas clases).

Taller | Recursos sobre PyTorch

- [Capas lineales](#).
- [Funciones de activación](#).
- [Normalización](#).
- [Dropout](#).
- [Data augmentation](#) (por ejemplo, [RandomHorizontalFlip](#)). Importante: ¡pocas aumentaciones son aplicables a imágenes tan pequeñas!

Sugerencia: enfocarse en los dos primeros.

Taller | ¡Que empiece el juego!

Recordemos que el objetivo es **probar varios experimentos y comparar su performance**.

A modo de referencia, la cátedra obtuvo un **accuracy de test del 75,33%**, en un tiempo similar a la duración de este taller, con las mismas herramientas que las hoy presentadas y con la siguiente **configuración** para los experimentos:

- `epochs = 500`,
- `subset_train = 1200` y
- `subset_val = 200`.

Para ello, se seleccionó un modelo que, en **validación**, había logrado un accuracy de 81% en el subset de validación de 200 muestras, con la configuración descrita arriba.

Pueden **variar** también estos hiperparámetros, si lo consideran conveniente.

Taller | Resoluciones

¿Cómo fue? ¿Alguien logró obtener un accuracy mayor al 75,33%?

Veamos `td6-p15-c-resoluciones.ipynb`.